

Revisão do Dia 3

```
>>> # Listas
```

```
>>> lst: list[int] = [2, 1, 4]
```

```
>>> lst.pop()
```

```
4
```

```
>>> lst.append(5)
```

```
>>> lst
```

```
[2, 1, 5]
```

```
>>> lst.insert(2, 8)
```

```
>>> lst
```

```
[2, 1, 8, 5]
```

```
>>> len(lst)
```

```
4
```

```
>>> # Laços de Repetição
```

```
>>> # for i in lista
```

```
>>> soma: int = 0
```

```
>>> for num in lst:
```

```
...     print(num)
```

```
...     soma += num
```

```
...
```

```
2
```

```
1
```

```
8
```

```
5
```

```
>>> soma
```

```
16
```

revisao_dia_3.py x

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

```
>>> # for i in range
>>> soma = 0
>>> for i in range(len(lst)):
...     print(lst[i])
...     soma += lst[i]
...
2
1
8
5
>>> soma
16
```

```
>>> # while *condição*
>>> soma = 0
>>> while i < len(lst):
...     print(lst[i], i)
...     soma += num
...     i += 1
...
2 0
1 1
8 2
5 3
>>> soma
16
```

dia_4.py x

Dia 4 – Funções e Tipos de Dados Personalizados

'''

Nós já vimos anteriormente como utilizar laços de repetição, mas como faríamos para reaproveitar uma mesma lógica em diferentes partes do programa de forma eficiente?

Vamos supor que o operador de multiplicação (*) não existisse na linguagem Python e que precisássemos fazer uma multiplicação por 2 em nosso programa.

Como fazer isso?

'''

dia_4.py x

'''

Bem, esse problema é bem simples de resolvermos. Como a multiplicação é uma sequência de somas e queremos multiplicar um número por 2, basta somarmos ele com ele mesmo.

```
>>> n: int = int(input("Digite um número inteiro: "))
```

```
>>> dobro: int = n + n
```

```
>>> print("O dobro de " + str(n) + " é " + str(dobro))
```

Certo, agora imagine que a gente precise realizar diversas multiplicações que sejam maiores que o dobro. Provavelmente, ficar escrevendo $n + n + n + \dots + n$ o tempo todo pode ser meio incômodo e não é muito semântico, ou seja, não fica claro que o objetivo é calcular uma multiplicação.

Sendo assim, como resolver esse problema?

'''

Funções

'''

Uma forma de resolvermos esse problema é com funções!

O conceito de função resume-se a um pedaço de código ao qual damos um nome e que podemos reutilizar quantas vezes quisermos no nosso programa.

Separar o nosso programa em múltiplas funções possui diversos benefícios:

- Reduz a quantidade de código repetido.
- Facilita a leitura e a manutenção do código, além de agilizar a busca e correção de bugs.
- Encapsula as partes do nosso programa, separando-o em pequenos trechos isolados que garantem maior estabilidade e menor chance de erros.

'''

Estrutura e Parâmetros

'''

Uma função em Python é definida usando a palavra reservada `def` e possui as seguintes partes: nome, parâmetros, tipo de retorno e o corpo da função. Os parâmetros são as variáveis que declaramos para receber os valores de entrada que a função precisa para trabalhar.

```
>>> def nome_funcao(parametro1: tipo1, parametro2: tipo2) -> tipo_retorno:
```

```
...     # * corpo função *
```

```
...     return valor_retorno
```

Após a execução da lógica no corpo da função, usamos a palavra `return` para retornar o valor final que a nossa função calculou.

'''

funcoes.py x

'''

Vejamos como podemos implementar a função de multiplicar que mencionamos no início:

'''

```
>>> def multiplica(n: int, vezes: int) -> int:
```

```
...     multiplicado: int = 0
```

```
...     for i in range(vezes):
```

```
...         multiplicado += n
```

```
...     return multiplicado
```

```
...
```

funcoes.py x

```
1
2
3 >>> # Agora podemos multiplicar um número da seguinte forma:
```

```
4 >>> n: int = int(input("Digite o número que deseja multiplicar: "))
```

```
5 >>> vezes: int = int(input("Por quantas vezes deseja multiplicar: "))
```

```
6 >>> # Multiplicando o valor de n pelo valor de vezes
```

```
7 >>> multiplicado: int = multiplica(n, vezes)
```

```
8 >>> print(str(n) + " X " + str(vezes) + " = " + str(multiplicado))
```

```
9
10 '''
```

```
11 Mas espera, nós já não usamos n, vezes e multiplicado como nome de variáveis
12 antes na função? Isso não pode gerar algum problema?
```

```
13 '''
14
15
```


Escopo de Variáveis

'''

A resposta é não!

Existe um conceito chamado escopo de variáveis, que garante que nenhuma variável dentro de uma função possa ser acessada fora dela.

Isso significa que as variáveis que definimos na função não podem ser usadas fora dela, sendo assim variáveis com o mesmo nome dentro e fora de uma função são completamente diferentes!

'''

funcoes.py x

'''

Exemplo: criar uma função que receba o nome de uma pessoa e exiba "bom dia" para essa pessoa.

Esse é um caso específico onde uma função não possui valor de retorno, pois não há a necessidade de se retornar nada, apenas exibir a mensagem de bom dia usando print.

Como definimos o tipo de retorno e o que colocamos no return da nossa função?

Simples: nada!

'''

funcoes.py x

```
1
2 >>> def bom_dia(nome: str) -> None:
3     ...     print("Bom dia, " + nome + "!")
4     ...     return
5
6     '''
7 Melhor ainda, nem precisamos colocar um return na nossa função, já que ele
8 não será usado.
9     '''
10
11 >>> def bom_dia(nome: str) -> None:
12     ...     print("Bom dia, " + nome + "!")
13 >>> nome_usuario = input("Digite o seu nome: ")
14 >>> bom_dia(nome_usuario)
15
```

funcoes.py x

```
1  
2 '''  
3 Vamos criar uma nova função. Dessa vez, ela deve receber um tempo em segundos  
4 e retornar seu equivalente em horas, minutos e segundos, de forma que minutos  
5 e segundos nunca sejam maiores ou iguais a 60.
```

```
6  
7 Problema: como vamos retornar 3 valores distintos em uma única função?
```

```
8  
9 Uma possível solução seria retornar eles em uma lista  
10 tempo: list[int] = [horas, minutos, segundos]!
```

```
11  
12 Vamos tentar.
```

```
13 '''  
14  
15
```

funcoes.py x

```
1
2
3 >>> def converte_tempo(tempo: int) -> list[int]:
4     ...     # Calcula horas (1 hora = 3600 segundos)
5     ...     horas: int = tempo // 3600
6     ...     tempo = tempo % 3600 # remove as horas já separadas
7     ...     # Calcula minutos (1 minutos = 60 segundos)
8     ...     minutos: int = tempo // 60
9     ...     # 0 restante do tempo são os segundos
10    ...     segundos: int = tempo % 60
11    ...     return [horas, minutos, segundos]
12    ...
13
14
15
```


funcoes.py x

'''

Nossa solução parece adequada?

Não, pois o objetivo de uma lista é armazenar elementos de uma mesma natureza, ou seja, que representem a mesma coisa.

Entretanto, o retorno da nossa função é uma lista de 3 coisas diferentes (horas, minutos, segundos) que, apesar de serem unidades de tempo, são formas distintas de representá-lo. Em resumo, não são a mesma coisa, logo não é adequado guardá-los em uma mesma lista.

Como podemos resolver esse problema?

'''



tipos_dados_compostos.py x

Dados Compostos

'''

Um tipo composto é um tipo de dado personalizado que podemos criar e que pode unir pequenos dados para representar um dado maior.

Por exemplo, no nosso caso, podemos criar um tipo composto chamado Tempo que é composto (daí o nome) por horas, minutos e segundos.

Um tipo composto em Python é chamado de dataclass e é necessário importar essa funcionalidade para dentro do nosso programa no início dele. A linha é simples:

```
>>> from dataclasses import dataclass
```

'''

tipos_dados_compostos.py x

'''

Para criar nosso tipo composto podemos seguir o seguinte modelo:

```
>>> @dataclass
```

```
... class NomeDoTipo: # Nomes de tipos são escritos em PascalCase
```

```
...     campo1: tipo1
```

```
...     campo2: tipo2
```

```
...     ...
```

Cada campo funciona como se fosse uma variável interna do tipo e ela deve ser obrigatoriamente preenchida quando criarmos ele.

Um tipo composto é um tipo de dado personalizado que podemos criar e que pode unir pequenos dados para representar um dado maior.

'''



tipos_dados_compostos.py x

1
2
3 # Para usarmos o tipo composto que criamos podemos fazer da seguinte forma:
4

5 >>> NomeDoTipo(valor1, valor2, valor3, ...)

6 >>> # Salvando em uma variável nova

7 >>> minha_variavel: NomeDoTipo = NomeDoTipo(valor1, valor2, valor3, ...)

8
9 '''

10 Observação importante: a ordem dos valores que inserimos quando criamos nosso
11 dado deve ser a mesma ordem de quando criamos o tipo.

12 '''
13
14
15



tipos_dados_compostos.py x

1
2
3 # Vamos adaptar nossa função de converter tempo para usar tipo composto:
4

5 >>> from dataclasses import dataclass

6 >>> @dataclass

7 ... class Tempo:

8 ... horas: int

9 ... minutos: int

10 ... segundos: int

11 ...
12
13
14
15

tipos_dados_compostos.py x

```
1
2
3 >>> def converte_tempo(tempo: int) -> list[int]:
4     ...     # Calcula horas (1 hora = 3600 segundos)
5     ...     horas: int = tempo // 3600
6     ...     tempo = tempo % 3600 # remove as horas já separadas
7     ...     # Calcula minutos (1 minutos = 60 segundos)
8     ...     minutos: int = tempo // 60
9     ...     # 0 restante do tempo são os segundos
10    ...     segundos: int = tempo % 60
11    ...
12    ...     return Tempo(horas, minutos, segundos)
13    ...
14
15
```



tipos_dados_compostos.py x

1

2

3

4

'''

5

Certo, mas como podemos consultar os dados internos de um tipo composto?

6

7

Para fazer isso, podemos usar o nome da variável + ponto (.) + nome do campo:

8

9

>>> variavel_do_tipo.nome_do_campo

10

'''

11

12

13

14

15

tipos_dados_compostos.py x

'''

Vamos fazer uma nova função que recebe um tempo usando o tipo Tempo que criamos e que exibe uma mensagem personalizada para o usuário.

'''

```
>>> def exibe_tempo(tempo: Tempo):
```

```
...     horas: int = tempo.horas
```

```
...     minutos: int = tempo.minutos
```

```
...     segundos: int = tempo.segundos
```

```
...     print(str(horas) + " horas, " + str(minutos) + " minutos e " +
```

```
...         str(segundos) + " segundos")
```

```
...
```


tipos_dados_compostos.py x

'''

Por fim, também podemos mudar os valores de um campo específico da variável que contém o dado composto.

Para isso, fazemos a mesma coisa que no acesso ao dado (nome da variável + ponto + nome do campo)

'''

```
>>> meu_tempo: Tempo = Tempo(8, 15, 20)
```

```
>>> exibe_tempo(meu_tempo) # Exibe: 8 horas, 15 minutos e 20 segundos
```

```
>>> meu_tempo.horas = 4
```

```
>>> exibe_tempo(meu_tempo) # Exibe: 4 horas, 15 minutos e 20 segundos
```

```
>>> meu_tempo.minutos = 5
```

```
>>> meu_tempo.segundos = 0
```

```
>>> exibe_tempo(meu_tempo) # Exibe: 4 horas, 5 minutos e 0 segundos
```



tipos_dados_compostos.py x

1

2

3

'''

4

Continuando a prática de uso de funções, vamos criar uma função que recebe a

5

cor atual do semáforo e retorna a próxima cor.

6

- Vermelho -> Verde

7

- Verde -> Amarelo

8

- Amarelo -> Vermelho

9

10

De que forma podemos representar a cor da entrada e da saída da função?

11

'''

12

13

14

15

tipos_dados_compostos.py x

```
1
2 >>> # Uma string com o nome da cor deve servir:
3 >>> def proxima_cor(cor_atual: str) -> str:
4 ...     proxima_cor: str
5 ...     if cor_atual == "vermelho":
6 ...         proxima_cor = "verde"
7 ...     elif cor_atual == "verde":
8 ...         proxima_cor = "amarelo"
9 ...     else: # Cor atual é amarelo
10 ...         proxima_cor = "vermelho"
11 ...     return proxima_cor
12 ...
13
14
15
```




tipos_dados_compostos.py x

1
2 '''

3 Será que usar uma string para representar a cor foi uma boa decisão?

4
5 Imagine que por um erro de digitação a pessoa que vai usar a função sem
6 querer digite "vremelho" ao invés de "vermelho".

7
8 Ou pior, que sem querer a pessoa que faz a função fez com que ela retornasse
9 "vere" no lugar de "verde" porque o teclado falhou e ela não percebeu.

10
11 Esse tipo de problema pode gerar erros silenciosos no nosso programa que
12 podem nos custar uma quantidade de tempo muito maior do que queríamos para
13 corrigir um bug bobo desses.

14 '''
15



tipos_dados_compostos.py x

1
2 '''

3 Além disso, ter uma função que recebe uma string e retorna outra não é nada
4 semântico para nossa função, porque uma string é uma cadeia qualquer de
5 caracteres.

6
7 Porém, nossa função só trabalha com 3 strings específicas: "vermelho",
8 "verde" e "amarelo".

9
10 Seria muito melhor se pudéssemos dizer que a entrada e a saída da nossa
11 função são apenas cores.

12
13 Como podemos corrigir esses dois problemas do nosso código?

14 '''
15



tipos_dados_enumerados.py x

Dados Enumerados

'''

Um tipo enumerado é um tipo de dado personalizado que definimos valores específicos que ele pode assumir.

Para exemplificar, pense no tipo bool, quais valores uma variável desse tipo pode ter?

- Apenas True e False, nada mais.

Para criarmos nosso tipo enumerado também precisamos importar algumas funcionalidades adicionais para o nosso código:

```
>>> from enum import Enum, auto
```

```
>>> # Perceba que o segundo Enum tem letra maiúscula!
```

'''

tipos_dados_enumerados.py x

```
1
2 >>> # Agora podemos criar o nosso próprio tipo enumerado da seguinte forma:
3 >>> from enum import Enum, auto
4 >>> class MeuTipoEnumerado(Enum):
5     ...     VALOR1 = auto()
6     ...     VALOR2 = auto()
7     ...     VALOR3 = auto()
8
9     '''
10 Cada valor que colocarmos no nosso tipo é um valor que ele poderá assumir.
11
12 Por convenção, escrevemos os nomes dos valores de um tipo enumerado em letra
13 maiúscula.
14     '''
15
```

tipos_dados_enumerados.py x

'''

Por fim, podemos usar o nosso novo tipo fazendo nome do tipo + ponto (.) +
NOME DO VALOR (similar ao tipo composto):

'''

>>> # Associando a uma variável

>>> minha_variavel: MeuTipoEnumerado = MeuTipoEnumerado.VALOR

>>> # Fazendo comparação

>>> if variavel == MeuTipoEnumerado.VALOR:

... * código *

tipos_dados_enumerados.py x

Vamos aplicar o que aprendemos na nossa função de cores do semáforo:

```
>>> from enum import Enum, auto
```

```
>>> class Cor(Enum):
```

```
...     VERMELHO = auto()
```

```
...     AMARELO = auto()
```

```
...     VERDE = auto()
```

```
...
```

```
>>> def proxima_cor(cor_atual: Cor) -> Cor:
```

```
...     proxima_cor: Cor
```

```
...     if cor_atual == Cor.VERMELHO:
```

```
...         proxima_cor = Cor.VERDE
```

```
...     elif cor_atual == Cor.VERDE:
```

```
...         proxima_cor = Cor.AMARELO
```

```
...     else: # Cor atual é amarelo
```

```
...         proxima_cor = Cor.VERMELHO
```

```
...     return proxima_cor
```

```
...
```

tipos_dados_enumerados.py x

'''

Dica: Se precisarmos do nome de um valor de um tipo enumerado como string,
podemos usar .name() após o valor para obtê-la:

'''

```
>>> cor_atual: Cor = Cor.VERMELHO
```

```
>>> print("A cor atual é " + cor_atual.name())
```

```
"A cor atual é VERMELHO"
```

```
>>> print("Eu gosto da cor " + Cor.VERDE.name())
```

```
"Eu gosto da cor VERDE"
```

exercicio.py x

Jogo RPG

'''

Você é um game dev de uma empresa de jogos onde está sendo desenvolvido o jogo "Ilha de Esgard", um jogo RPG de "dark fantasy", e você ficou responsável por desenvolver o código do jogador!

Você deve criar um jogador, que possui uma classe (que pode ser mago, guerreiro, arqueiro ou tanque), nível, experiência (que pode aumentar e ao chegar em um certo ponto zera para aumentar o nível) e vida.

Faça funções para criar um jogador, aumentar a experiência, para aumentar o nível quando ela passar de um ponto limite, e uma função para dar dano no jogador.

'''

agradecimentos.py x

MUITO OBRIGADO!

Pesquisa de Feedback :)

